

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ МИСИС»

Институт компьютерных наук
Кафедра Инженерной кибернетики

Курсовая работа
по дисциплине «Алгоритмы дискретной математики»

Компьютерная сборка кубика Рубика

Выполнили:

Лехтерева Дарья Дмитриевна БПМ-24-1,
Катков Фёдор Станиславович БПМ-24-1,
Евстигнеева Арина Николаевна БПМ-24-1,
Ломакина Олеся Александровна БПМ-24-1

Москва, 2026

Содержание

1 Введение	2
1.1 Актуальность темы	2
1.2 Цель работы	2
1.3 Задачи работы	2
2 Теоретическая часть	2
2.1 Аппарат теории групп и перестановок	2
2.2 Математическая модель Кубика Рубика	3
2.3 Алгоритм разложения на независимые циклы	4
3 Ход работы	4
3.1 Структура данных и представление состояния	4
3.2 Алгоритм применения перестановки	5
3.3 Алгоритм разложения состояния на независимые циклы	5
3.4 Двухфазный метод поиска и координатные пространства	5
3.5 Формирование баз данных шаблонов	6
3.6 Оптимизация пространства поиска и алгоритм IDA*	6
3.7 Локальная оптимизация результирующей последовательности	9
3.8 Оценка сложности алгоритма	9
3.8.1 Пространственная сложность структуры состояния куба	10
3.8.2 Анализ процедуры Apply-Permutation	10
3.8.3 Анализ процедуры Find-Disjoint-Cycles	10
3.8.4 Анализ построения таблиц переходов и баз данных шаблонов	11
3.8.5 Анализ алгоритма направленного поиска IDA*	11
4 Компьютерная реализация	12
4.1 Программная архитектура и интерфейсы классов	12
4.2 Обоснование выбора инструментария разработки	14
5 Заключение	14
Список литературы	15

1 Введение

1.1 Актуальность темы

Головоломка "Кубик Рубика" представляет собой достаточно сложную комбинаторную задачу, получившую широкое распространение. Популярность этой головоломки аналогична популярности, которую имела примерно за сто лет до этого другая знаменитая "игра в пятнадцать", изобретенная создателем головоломок и шахматным композитором Самуэлем Лойдом. Правда, в отличие от кубика тот вариант "игры в пятнадцать", который предлагал Лойд, не имел решения. С 1980 по 1982 год было продано свыше 100 миллионов кубиков. Сложность сборки игрушки вызвала к жизни поток специальных изданий по этой проблеме. В СССР журнал "Наука и жизнь" во многих номерах уделял место Кубику Рубика. Наконец, в июне 1982 г. в Будапеште состоялся мировой чемпионат по сборке кубика.

Существует несколько алгоритмов решения этой головоломки. Наиболее известен метод "последовательной сборки" позволяющий собрать кубик, в среднем, за сто оборотов. Компьютерная программа, работающая по этому алгоритму, собирает кубик примерно за 50 шагов. Проблема поиска оптимального решения этой задачи является достаточно интересной и сложной. Известный американский математик Дональд Эрвин Кнут в своей книге "Введение в курс математики кубика Рубика" доказал, что сборку можно осуществить не более, чем за 22 вращения.

1.2 Цель работы

Целью данной курсовой работы является исследование математической модели головоломки "Кубик Рубика", используя теории групп, и разработка программного алгоритма, способного привести кубик из произвольного запутанного состояния в исходное, или правильное, состояние.

1.3 Задачи работы

Для решения поставленной задачи, построим план со следующими задачами:

1. Построим математическую модель кубика Рубика, описав все состояния головоломки и вращения её граней языком Алгебры
2. Разработаем алгоритм перебора для сборки кубика, используя формулы теории групп
3. Выполним программную реализацию алгоритма и наглядно продемонстрируем процесс компьютерной сборки головоломки на экране дисплея

2 Теоретическая часть

2.1 Аппарат теории групп и перестановок

Для начала разберем необходимый аппарат теории групп и перестановок, который будет использоваться для построения математической модели кубика Рубика. Каждое состояние головоломки можно рассматривать как определенную расстановку 54 цветных сегментов. Любое вращение грани в таком случае является перестановкой — взаимно однозначным отображением множества сегментов на себя.

Пусть $X = \{1, 2, \dots, n\}$ — множество из $n = 54$ элементов, что соответствует общему количеству цветных сегментов на гранях куба. Перестановка — это взаимно однозначная функция вида $f : X \rightarrow X$. Обычно перестановку изображают в виде матрицы из n столбцов и 2 строк:

$$S = \begin{pmatrix} 1 & 2 & 3 & \dots & n \\ f(1) & f(2) & f(3) & \dots & f(n) \end{pmatrix} \quad (1)$$

Немаловажным понятием является так называемая тождественная перестановка (ϵ), при которой каждый элемент перестановки переходит сам в себя, $f(i) = i, \forall i \in X$. Тождественная перестановка играет существенную роль в нашей задаче, так как она характеризует начальное положение кубика Рубика. Если S — перестановка, тогда обратная перестановка, S^{-1} , будет возвращать систему в исходное положение.

Будем описывать последовательность поворотов граней куба как композицию (произведение) соответствующих граням перестановок. Также для разработки алгоритма сборки кубика ключевую роль будет играть свойство разложимости: любую перестановку можно представить в виде композиции взаимно простых циклов, где цикл - это некоторое подмножество элементов, которые при последовательном применении перестановки переходят друг в друга по кругу.

Одной из важных характеристик перестановки является порядок перестановки. Порядок перестановки - это такое наименьшее натуральное число k , что:

$$S^k = \underbrace{S \circ S \circ \dots \circ S}_k = \epsilon \quad (2)$$

Наличие порядка гарантирует, что при многократном повторении одной и той же последовательности поворотов, кубик рано или поздно вернется в исходное состояние. Также стоит учитывать, что множество всех состояний кубика образует группу, в которой выделим систему образующих - минимальный набор базовых перестановок, с помощью композиции которых можно будет получить любое состояние кубика. В случае, если разложение на циклы окажется достаточно сложным, наш алгоритм сможет использовать формулы выражения перестановки через системы образующих элементов:

$$S = a_1^{k_1} \circ a_2^{k_2} \circ \dots \circ a_m^{k_m}, \text{ где } a_i - \text{повороты из системы образующих} \quad (3)$$

2.2 Математическая модель Кубика Рубика

Будем рассматривать наш кубик как систему, состояние которой будет определяться взаимным расположением его элементов. Наш куб состоит из 26 малых кубиков и 54 цветных сегментов.

Для однозначной идентификации сегментов введем трехмерную систему координат O_{xyz} . Центры граней будут неподвижны, что позволит нам закрепить оси относительно цветов центральных сегментов:

- **Ось x:** будет выходить из синего сегмента и входить в зеленый;
- **Ось y:** будет выходить из желтого сегмента и входить в белый;
- **Ось z:** будет выходить из коричневого сегмента и входить в красный;

Порядок граней определим следующим образом: если мы посмотрим на куб с положительного/отрицательного направления осей, то мы получим последовательность граней $(X, Z, -X, -Z, Y, -Y)$, номера которых соответственно $(1, 2, 3, 4, 5, 6)$.

В свою очередь, каждая из 6 граней куба содержит 9 сегментов, которые будут логически нумероваться слева направо и сверху вниз. Разобьем куб на плоскости, или же слои из 9 кубиков, перпендикулярные осям. Плоскость обозначим некоторым индексом $j \in \{1, 2, 3\}$, где нумерация будет вестись против направления соответствующей оси поворота.

Перейдем к основной части - действие над кубом. Обозначим действие над кубом как $\varphi_j^i(\alpha)$, где $i \in \{x, y, z\}$ - ось поворота, $j \in \{1, 2, 3\}$ - номер плоскости, α - угол поворота. Тогда, мы получим 27 базовых поворотов. Общее количество базовых поворотов обусловлено наличием 3 координатных осей, 3 параллельных им плоскостей и 3 возможных вариантов поворота для каждой плоскости соответственно $(\frac{\pi}{2}, \pi, -\frac{\pi}{2})$. При этом нам стоит учесть, что повороты по часовой стрелке будут считаться положительными, а против - отрицательными. Также учтем, что любой сложный поворот мы сможем выразить через базовый поворот Φ :

$$\Phi_i^{-90} = (\Phi_i^{90})^{-1}, \Phi_i^{180} = \Phi_i^{90} \circ \Phi_i^{90} \quad (4)$$

Любое состояние куба в свою очередь представляется в виде перестановки (перестановки были описаны в разделе выше) - взаимно однозначной функции $f: \{1, \dots, 54\} \rightarrow \{1, \dots, 54\}$.

Как итог, задачу компьютерной сборки кубика сформулируем как поиск некоторой последовательности поворотов $\varphi_1, \varphi_2, \dots, \varphi_k$, композиция которой с начальной перестановкой S будет приводить к тождественной перестановке $\epsilon: S \circ \varphi_1 \circ \varphi_2 \circ \dots \circ \varphi_k = \epsilon$

2.3 Алгоритм разложения на независимые циклы

Для нахождения той самой последовательности базовых поворотов $\varphi_j^i(\alpha)$, приводящих систему к тождественной перестановке ϵ , нам необходимо разложить перестановки на циклы. Любую перестановку S , описывающее текущее состояние 54 сегментов, мы представим в виде произведения взаимно простых циклов:

$$S = S_1 \circ S_2 \circ \dots \circ S_m, \text{ где } S_k - \text{перестановка, образующая отдельный цикл} \quad (5)$$

Для восстановления исходного состояния куба мы применим обратную последовательность действий для каждого цикла S_k , получая по итогу:

$$S \circ (S_1^{-1} \circ S_2^{-1} \circ \dots \circ S_m^{-1}) = \epsilon \quad (6)$$

В свою очередь для самого алгоритма мы будем применять последовательность шагов вида:

Шаг 1: создадим массив "флагов" для того, чтобы отслеживать уже обработанные элементы

Шаг 2: возьмем первый непомеченный сегмент x , где $f(x) \neq x$

Шаг 3: последовательно определим позиции $x \rightarrow f(x) \rightarrow f(f(x)) \rightarrow f(f(f(x))) \rightarrow \dots$ ровно до тех пор, пока очередное отображение не вернет индекс, возвращающий к исходному x

Шаг 4: все сегменты, которые войдут в эту цепочку, мы пометим как обработанные, и сохраним их как цикл S_k . Повторим этот процесс для всех 54 элементов

Однако, сложность самого алгоритма не в шагах его выполнения, а в выражении самого обратного цикла S_k^{-1} через композицию базовых поворотов. Небольшое отступление: назовем подвижными точками 9 сегментов грани, на которую воздействует базовый поворот $\varphi_j^i(\alpha)$. Отсюда следует, что количество подвижных точек в базовом повороте превышает количество сегментов в S_k , а значит наш алгоритм компенсирует "лишние" перемещения путем композиции, при которой те самые подвижные точки могут превращаться в неподвижные:

$$S_k^{-1} = a_{j_1}^{i_1} \circ a_{j_2}^{i_2} \circ \dots \circ a_{j_n}^{i_n}, \text{ где } a_j^i - \text{базовые повороты из системы образующих} \quad (7)$$

В дополнение нужно учесть, что в случае минимизации влияния на уже собранные сегменты и точечного взаимодействия на конкретный цикл мы будем применять формулу коммутация:

$$\Phi = b \circ a \circ b^{-1}, \text{ где } a \text{ и } b - \text{циклы, состоящие из избыточного множества подвижных точек} \quad (8)$$

3 Ход работы

3.1 Структура данных и представление состояния

Для представления математической модели нам необходимо выбрать структуру данных, которая сможет эффективно обрабатывать перестановки.

Как мы уже знаем, наш куб состоит из 26 небольших кубиков (27 небольшой кубик не считаем, так как он находится в центре), и 54 цветных сегментов. За основу представления состояния кубика возьмем одномерный массив $A[1 \dots 54]$, где i - индекс, который соответствует фиксированной позиции на грани куба, а $A[i]$ будет хранить идентификатор цвета сегмента, находящегося в этой позиции в данный момент времени. Тождественная перестановка ϵ в свою очередь будет представлена отсортированным массивом, где цвета будут сгруппированы по 9 элементов, которые соответствуют каждой грани.

Помимо этого, для хранения 27 базовых поворотов мы будем использовать хэш-таблицу, где ключом будет выступать строковое обозначение поворота (например, **R**, **U**, **F2**, **U'** и так далее), а значением - массив-перестановка $P[1 \dots 54]$, который в свою очередь описывает, куда перемещается каждый сегмент.

3.2 Алгоритм применения перестановки

Базовая операция нашей реализации - это применение поворота к текущему состоянию. Мы будем достигать этого путем применения массива-перестановки P к массиву состояния. Так как разменность массива фиксирована, $n = 54$, операция будет работать за линейное время, которое в рамках нашей задачи мы будем считать за константное время $O(1)$.

Algorithm 1 Применение перестановки к состоянию куба

```

1: procedure APPLY-PERMUTATION(State, P)
2:    $N \leftarrow \text{State.length}$ 
3:    $\text{New\_State}[1 \dots N] \leftarrow$  пустой список
4:   for  $i = 1$  to  $N$  do
5:      $\text{New\_State}[i] \leftarrow \text{State}[P[i]]$ 
6:   end for
7:   return  $\text{New\_State}$ 
8: end procedure

```

3.3 Алгоритм разложения состояния на независимые циклы

Основой нашего метода решения является есть утверждение о том, что **любую перестановку можно разложить на произведение взаимно-простых циклов (композиция)**. Чтобы программа могла найти обратные циклы и выразить их через базовые повороты, нам стоит проанализировать текущее состояние массива A и сравнить его с целевым состоянием E (тождественное состояние). Для этого мы реализуем метод поиска циклов, который будет обходить массив состояния и группировать элементы, которые находятся не на своих местах.

Algorithm 2 Поиск непересекающихся циклов

```

1: procedure FIND-DISJOINT-CYCLES(CurrentState, TargetState)
2:    $Cycles \leftarrow$  пустой список
3:    $visited[1 \dots 54] \leftarrow$  массив, заполненный значением FALSE
4:   for  $i \leftarrow 1$  to  $54$  do
5:     if  $visited[i] == \text{TRUE}$  or  $\text{CurrentState}[i] == \text{TargetState}[i]$  then
6:       continue
7:     end if
8:      $current\_cycle \leftarrow$  пустой список
9:      $current\_index \leftarrow i$ 
10:    while  $visited[current\_index] == \text{FALSE}$  do
11:       $visited[current\_index] \leftarrow \text{TRUE}$ 
12:      Добавить  $current\_index$  в  $current\_cycle$ 
13:       $current\_index \leftarrow \text{INDEX-OF}(\text{TargetState}, \text{CurrentState}[current\_index])$ 
14:    end while
15:    Добавить  $current\_cycle$  в  $Cycles$ 
16:  end for
17:  return  $Cycles$ 
18: end procedure

```

3.4 Двухфазный метод поиска и координатные пространства

Для обхода вычислительной сложности, связанной с колоссальным объемом пространства состояний кубика ($\approx 4.33 \times 10^{19}$), в работе был реализован оптимизированный двухфазный алгоритм поиска (известный как алгоритм Коцебмы). Вместо построения единого сквозного маршрута от произвольного состояния к тождественному, процесс поиска разделяется на два последовательных этапа, каждый из которых оперирует своей подгруппой перестановок.

Для эффективного масштабирования физическое представление кубика (массив из 54 сегментов) отображается в специальное координатное представление. Каждая координата является целым числом, отражающим определенный аспект подмножества элементов:

1. Координаты первой фазы:

- Ориентация угловых элементов ($co \in [0, 2186]$) — кодирует разворот 8 углов;
- Ориентация реберных элементов ($eo \in [0, 2047]$) — кодирует разворот 12 ребер;
- Положение ребер среднего слоя ($slice \in [0, 494]$) — определяет сочетание позиций для 4 ребер среднего слоя (UD-slice).

Цель первой фазы - привести произвольное состояние куба к промежуточной подгруппе, в которой все ребра будут ориентированы правильно, углы будут ориентированы правильно, а ребра среднего слоя будут находиться на своих позициях (пусть и не на своих местах). Промежуточная группа: $G_1 = \langle U, D, R2, L2, F2, B2 \rangle$

2. Координаты второй фазы:

- Перестановка углов ($cp \in [0, 40319]$) — определяет точное взаимное расположение угловых элементов;
- Перестановка ребер верхнего и нижнего слоев ($ep_ud \in [0, 40319]$) — задает размещение оставшихся 8 ребер;
- Перестановка ребер среднего слоя ($ep_slice \in [0, 23]$) — задает финальный порядок 4 ребер внутри среднего слоя.

На второй фазе мы допускаем использование только подмножества из 10 базовых поворотов подгруппы G_1 , что гарантирует сохранение инвариантов, достигнутых на первом этапе, и приводит куб к тождественному состоянию.

Для мгновенного вычисления переходов между координатами под воздействием вращений мы предварительно генерируем таблицы переходов (Transition Tables), выполняющие роль функций вида $f(Coordinate, Move) \rightarrow NextCoordinate$.

3.5 Формирование баз данных шаблонов

В качестве допустимых эвристических функций $h(x)$ для алгоритма направленного перебора мы используем предварительно вычисленные базы данных шаблонов (Pattern Databases, PDB). Они формируются путем обратного поиска в ширину (BFS) из целевого состояния до момента исчерпания соответствующего координатного подпространства.

Поскольку точное совместное распределение всех координат требует слишком больших объемов памяти, эвристика декомпозируется на независимые составляющие. Для первой фазы вычисляются две базы данных, связывающие ориентации с положением ребер среднего слоя: PDB_{co_slice} и PDB_{eo_slice} . Для второй фазы строятся базы данных перестановок: PDB_{cp_slice} и $PDB_{ep_ud_slice}$.

Финальные эвристические оценки для каждого состояния определяются как точные максимумы из соответствующих баз данных, что гарантирует свойство допустимости (отсутствие переоценки стоимости пути) и консистентности:

$$h_1(state) = \max(PDB_{co_slice}[co, slice], PDB_{eo_slice}[eo, slice]) \quad (9)$$

$$h_2(state) = \max(PDB_{cp_slice}[cp, ep_slice], PDB_{ep_ud_slice}[ep_ud, ep_slice]) \quad (10)$$

3.6 Оптимизация пространства поиска и алгоритм IDA*

Поиск последовательности поворотов выполняется модифицированным алгоритмом поиска с итерационным углублением (IDA*). Данный подход объединяет оптимизацию по памяти, свойственную поиску в глубину, со свойством гарантированного нахождения кратчайшего пути, присущим алгоритму A*.

Для исключения заведомо избыточных ветвлений графа вычислений (пространства поиска) в алгоритм интегрированы правила отсечения по симметрии и коммутативности:

- **Запрет повторений:** Мы не допускаем последовательное вращение одной и той же грани (например, ход R после R , $R2$ или R' бессмысленен);
- **Иерархия противоположных граней:** При вращении противоположных параллельных граней (например, R и L) их порядок будет строго фиксироваться. Ход L после R разрешен, но ход R после L блокируется, так как данные операции коммутируют ($R \circ L = L \circ R$), и обход обеих ветвей привел бы к дублированию путей.

Ниже представлены псевдокоды взаимно рекурсивных процедур поиска первой и второй фаз, формирующих ядро вычислительного решения.

Algorithm 3 Поиск первой фазы алгоритма IDA*

```

1: procedure SEARCH-PHASE1(co, eo, slice, g, bound, last_face, path)
2:    $h \leftarrow \max(PDB_{co\_slice}[co, slice], PDB_{eo\_slice}[eo, slice])$ 
3:   if  $g + h > bound$  then
4:     return  $g + h$ 
5:   end if
6:   if  $g + h \geq min\_total\_length$  then
7:     return  $\infty$ 
8:   end if
9:   if  $h == 0$  then
10:     $state \leftarrow GET-STATE-FROM-PATH(path)$ 
11:     $cp \leftarrow GET-CP(state)$ ,  $ep\_ud \leftarrow GET-EP-UD(state)$ ,  $ep\_slice \leftarrow GET-EP-SLICE(state)$ 
12:     $bound2 \leftarrow \max(PDB_{cp\_slice}[cp, ep\_slice], PDB_{ep\_ud\_slice}[ep\_ud, ep\_slice])$ 
13:    while  $bound2 < min\_total\_length - g$  do
14:       $p2\_path \leftarrow$  пустой список
15:       $res2 \leftarrow SEARCH-PHASE2(cp, ep\_ud, ep\_slice, 0, bound2, last\_face, p2\_path)$ 
16:      if  $res2 == -1$  then
17:         $best\_solution \leftarrow path + p2\_path$ 
18:         $min\_total\_length \leftarrow g + bound2$ 
19:        if  $min\_total\_length \leq 22$  then
20:          return  $-2$ 
21:        end if
22:        break
23:      end if
24:       $bound2 \leftarrow res2$ 
25:    end while
26:  end if
27:   $min\_f \leftarrow \infty$ 
28:  for каждый  $m$  in  $0 \dots 17$  do
29:     $face \leftarrow m/3$ 
30:    if  $face == last\_face$  then
31:      continue
32:    end if
33:    if  $(face \oplus 1) == last\_face$  and  $face > last\_face$  then
34:      continue
35:    end if
36:    if  $h == 0$  then
37:      if  $face == 0$  or  $face == 1$  then
38:        continue
39:      end if
40:      if  $m \in \{7, 10, 13, 16\}$  then
41:        continue
42:      end if
43:    end if
44:     $PUSH(path, m)$ 
45:     $res \leftarrow SEARCH-PHASE1(Move_{co}[co, m], Move_{eo}[eo, m], Move_{slice}[slice, m], g + 1, bound, face, path)$ 
46:    if  $res == -2$  then
47:      return  $-2$ 
48:    end if
49:     $min\_f \leftarrow \min(min\_f, res)$ 
50:     $POP(path)$ 
51:  end for
52:  return  $min\_f$ 
53: end procedure

```

Algorithm 4 Поиск второй фазы алгоритма IDA*

```

1: procedure SEARCH-PHASE2( $cp, ep\_ud, ep\_slice, g, bound, last\_face, path$ )
2:    $h \leftarrow \max(PDB_{cp\_slice}[cp, ep\_slice], PDB_{ep\_ud\_slice}[ep\_ud, ep\_slice])$ 
3:   if  $g + h > bound$  then
4:     return  $g + h$ 
5:   end if
6:   if  $h == 0$  then
7:     return  $-1$ 
8:   end if
9:    $min\_f \leftarrow \infty$ 
10:  for  $idx \leftarrow 0$  to  $9$  do
11:     $m \leftarrow P2\_Moves[idx]$ 
12:     $face \leftarrow m/3$ 
13:    if  $face == last\_face$  then
14:      continue
15:    end if
16:    if  $(face \oplus 1) == last\_face$  and  $face > last\_face$  then
17:      continue
18:    end if
19:     $PUSH(path, m)$ 
20:     $res \leftarrow SEARCH\_PHASE2(Move_{cp}[cp, idx], Move_{ep\_ud}[ep\_ud, idx], Move_{ep\_slice}[ep\_slice, idx], g +$ 
1,  $bound, face, path)$ 
21:    if  $res == -1$  then
22:      return  $-1$ 
23:    end if
24:     $min\_f \leftarrow \min(min\_f, res)$ 
25:     $POP(path)$ 
26:  end for
27:  return  $min\_f$ 
28: end procedure

```

3.7 Локальная оптимизация результирующей последовательности

После успешного завершения обеих фаз поиска и объединения их путей, результирующий вектор ходов подвергается процедуре фильтрации и локального сжатия. Поскольку стык между первой и второй фазами может генерировать избыточные повороты одной и той же плоскости (например, последовательность $[U, U2]$, порожденная на разных этапах независимо), алгоритм осуществляет проход по результирующему стеку и схлопывает одноименные операции по правилам модульной арифметики вращений (по модулю 4):

$$\alpha_{res} = (\alpha_1 + \alpha_2) \bmod 4 \quad (11)$$

В случае, если суммарный угол поворота $\alpha_{res} = 0$, оба хода полностью аннулируются, уменьшая итоговую длину решения.

3.8 Оценка сложности алгоритма

Для проведения асимптотического анализа введем следующие базовые параметры:

- $N = 54$ - количество сегментов;
- $K = 27$ - количество базовых поворотов.

Однако, несмотря на то, что по условию задачи эти параметры константны, необходимость в них имеется - их параметризация позволит оценить масштабируемость предложенных алгоритмов. Теперь разберем каждый из выведенных нами выше алгоритма.

3.8.1 Пространственная сложность структуры состояния куба

Очевидно, что память, которая необходима для хранения текущего состояния куба составляет $O(N)$. Наша хэш-таблица базовых поворотов хранит K ключей, каждому из которых в свою очередь соответствует массив перестановок длины N . Отсюда следует, что общую пространственную сложность этой структуры данных можно оценить как:

$$S_{data}(N, K) = O(N) + O(K \cdot N) = O(K \cdot N) \quad (12)$$

С учетом того, что величины N и K являются константными, не трудно догадаться, что итоговое потребление памяти будет строго ограничено, и составляет $O(1)$.

3.8.2 Анализ процедуры Apply-Permutation

Как мы видим, процедура **Apply-Permutation** выделяет память под новый массив *New_State* длиной N , что требует $O(N)$ дополнительной памяти, значит, пространственную сложность мы можем оценить как $O(N)$. Внутренний цикл **for** выполняется ровно N раз, значит внутри цикла происходит обращение к элементам массива по индексу, что занимает $O(1)$. Несложно догадаться, что временная сложность применения одного поворота можно описать линейной функцией:

$$T_{apply}(N) = \sum_{i=1}^N [O(1)] = O(N) \quad (13)$$

3.8.3 Анализ процедуры Find-Disjoint-Cycles

Анализ данной процедуры будет требовать более детального рассмотрения вложенных циклов. Внешний цикл **for** итерируется по массиву N раз. Внутри него находится ещё цикл **while**, который обходит элементы, принадлежащие одному циклу перестановки. С одной стороны, наличие вложенного цикла может говорить о квадратичной сложности. С другой стороны, благодаря массиву *visited*, каждый из N элементов меняет свое состояние на **TRUE** ровно один раз за все время работы алгоритма.

Введем величину: пусть перестановка распадается на C независимых циклов, где L_j - длина j -го цикла; тогда сумма длин всех циклов будет ограничена общим числом элементов:

$$\sum_{j=1}^C [L_j] \leq N \quad (14)$$

В свою очередь, главным фактором, который влияет на итоговую сложность, будет являться процедура **Index-Of**. Если поиск элемента в целевом массиве *TargetState* реализован линейным подходом, то его сложность в худшем случае будет составлять $O(N)$. Из этого следует, что общее время работы будет составлять:

$$T_{linear_cycles}(N) = O(N) + \sum_{j=1}^C [L_j \cdot O(N)] = O(N) + O(N) \cdot \sum_{j=1}^C [L_j] = O(N^2) \quad (15)$$

Заметим, что для классического кубика Рубика, где $N = 54$, количество операций при $O(N^2)$ не может превышать ~ 2900 , что выполняется мгновенно. Однако, с точки зрения теории, алгоритм можно будет оптимизировать до линейного времени $O(N)$, если предварительно вычислить обратную перестановку для *TargetState*. В этом случае время работы **Index-Of** сократится до $O(1)$, и финальная асимптотика поиска циклов будет приведена к виду:

$$T_{optimal_cycles}(N) = O(N) + \sum_{j=1}^C [L_j \cdot O(1)] = O(N) + O(N) = 2 \cdot O(N) = O(N) \quad (16)$$

Следовательно, пространственная сложность процедуры **Find-Disjoint-Cycles** будет составлять $O(N)$, так как для работы будут требоваться массив *visited* и структура данных для хранения найденных циклов, суммарный размер которых не будет превышать N .

3.8.4 Анализ построения таблиц переходов и баз данных шаблонов

Основой производительности двухфазного алгоритма является предварительное вычисление таблиц. Введем параметр V , обозначающий объем пространства конкретной координаты (например, для ориентации углов $V_{co} = 3^7 = 2187$), и M — количество допустимых базовых ходов ($M_1 = 18$ для первой фазы, $M_2 = 10$ для второй).

Построение таблиц переходов осуществляется с помощью алгоритма поиска в ширину (BFS). Из каждого состояния генерируются все возможные переходы, что формирует временную сложность $O(V \cdot M)$.

Для баз данных шаблонов (PDB) размерность пространства состояний формируется как произведение мощностей комбинируемых подмножеств координат. Например, для базы PDB_{co_slice} размер пространства составит $V_{pdb} = V_{co} \cdot V_{slice}$. Обратный поиск в ширину обходит этот граф, требуя $O(\bar{V}_{pdb} \cdot M)$ времени на вычисление и $O(V_{pdb})$ памяти для хранения (используя 1 байт на каждое состояние).

3.8.5 Анализ алгоритма направленного поиска IDA*

Вычислительная сложность непосредственно процедуры сборки (поиска решения) зависит от двух ключевых факторов: эффективного коэффициента ветвления b графа состояний и целевой глубины поиска d . Временная сложность классического алгоритма IDA* в наихудшем сценарии экспоненциальна и выражается функцией:

$$T_{search}(d) = O(b^d) \quad (17)$$

На первой фазе изначальные 18 ветвей редуцируются в среднем до $b_1 \approx 13.5$. На второй фазе из 10 возможных ходов остается $b_2 \approx 7.3$. Применение точных и допустимых эвристик (PDB) радикально сокращает количество перебираемых узлов, отсекая бесперспективные поддеревья графа на ранних этапах, хотя теоретическая верхняя граница временной сложности остается $O(b^d)$.

Ключевым преимуществом выбранного нами метода IDA* перед алгоритмом A* выступает его пространственная сложность. Алгоритм не требует поддержки структур данных для хранения фронта поиска (открытого и закрытого множеств). Вместо этого IDA* опирается на поиск в глубину (DFS), выделяя память исключительно под хранение текущей ветви обхода - стека рекурсивных вызовов. Отсюда следует, что пространственная сложность процесса поиска имеет линейную зависимость от текущей глубины d :

$$S_{search}(d) = O(d) \quad (18)$$

Поскольку максимальная длина оптимального решения для кубика ограничена, потребление дополнительной памяти непосредственно во время решения головоломки пренебрежимо мало и оценивается константой $O(1)$.

4 Компьютерная реализация

4.1 Программная архитектура и интерфейсы классов

Для компьютерного моделирования математической структуры кубика и последующей реализации алгоритма его сборки, было принято решение выбрать объектно-ориентированную парадигму программирования (ООП). Архитектурное решение системы базируется на разделении ответственности между двумя ключевыми классами: `RubikCube` (модель данных) и `SolveRubikCube` (вычислительное ядро).

Класса `RubikCube` направлен на эффективное представление состояния куба в виде одномерного массива из 54 элементов и обеспечение интерфейса для манипуляции им. Описание интерфейса класса приведено ниже.

Листинг 1: Интерфейс класса модели кубика Рубика

```
1  class RubikCube {
2  public:
3      explicit RubikCube();
4      ~RubikCube() = default;
5
6
7      void applyMove(const std::string& name);
8      void applySequence(const std::vector<std::string>& moves);
9      void applyConjugation(const std::vector<std::string>& initial, const std::vector<std::
10         string>& original);
11     void applyCommutator(const std::vector<std::string>& first, const std::vector<std::
12         string>& second);
13
14     [[nodiscard]] bool solved() const;
15     [[nodiscard]] Cycles getCycles() const;
16     [[nodiscard]] const State& getStateArray() const;
17 private:
18     State state;
19     Moves table;
20     bool initialized;
21
22     void initMoveTable();
23     void generateDerivedMoves();
24     std::vector<std::string> invert(const std::vector<std::string>& sequence);
25 };
26
```

Класс `RubikCubeSolver` инкапсулирует в себе всю математическую логику. Его заголовочный файл представлен ниже.

Листинг 2: Интерфейс класса решателя

```

1  class RubikCubeSolver {
2
3  public:
4      explicit RubikCubeSolver(const RubikCube& initialCube);
5      std::vector<std::string> solve();
6  private:
7      RubikCube currentCube;
8
9      std::vector<int> best_solution;
10     std::vector<int> p2_path;
11     int min_total_length;
12
13     static bool tables_built;
14     static State base_moves[18];
15
16     static std::vector<std::vector<int>> move_co;
17     static std::vector<std::vector<int>> move_eo;
18     static std::vector<std::vector<int>> move_slice;
19     static std::vector<std::vector<int>> move_cp;
20     static std::vector<std::vector<int>> move_ep_ud;
21     static std::vector<std::vector<int>> move_ep_slice;
22
23     static std::vector<uint8_t> pdb_co_slice;
24     static std::vector<uint8_t> pdb_eo_slice;
25     static std::vector<uint8_t> pdb_cp_slice;
26     static std::vector<uint8_t> pdb_ep_ud_slice;
27
28     static int C_arr[13][13];
29     static int fact[9];
30
31     static int sticker_to_corner[CUBE_SIZE];
32     static int pos_to_ori[CUBE_SIZE];
33     static int sticker_to_edge[CUBE_SIZE];
34     static int pos_to_eori[CUBE_SIZE];
35     static int pos_to_edge[CUBE_SIZE];
36
37     static int get_co(const State& s);
38     static int get_eo(const State& s);
39     static int get_slice(const State& s);
40     static int get_cp(const State& s);
41     static int get_ep_ud(const State& s);
42     static int get_ep_slice(const State& s);
43     static int get_perm_rank(const std::vector<int>& perm);
44     void build_tables();
45     [[nodiscard]] State apply_move(const State& s, int m) const;
46     [[nodiscard]] std::vector<std::string> optimizeMoves(const std::vector<std::string>&
47         moves) const;
48     int search_p1(int co, int eo, int slice, int g, int bound, int last_face, std::vector<
49         int>& path);
50     int search_p2(int cp, int ep_ud, int ep_slice, int g, int bound, int last_face, std::
51         vector<int>& path);
52 };

```

4.2 Обоснование выбора инструментария разработки

В качестве языка программирования был выбран C++ (стандарт C++20). Данный выбор обусловлен следующими причинами:

- Высокая производительность при работе с низкоуровневыми структурами данных (массивами, векторами);
- Возможность строгой инкапсуляции математической модели в рамках ООП.

5 Заключение

В ходе выполнения данной курсовой работы было проведено исследование математической модели головоломки "Кубик Рубика" используя аппарат абстрактной алгебры, а также был разработан, оптимизирован и программно реализован эффективный алгоритм её автоматизированной сборки.

В рамках теоретической части работы на основе теории групп и теории перестановок была успешно построена строгая математическая модель кубика Рубика. Состояния головоломки были представлены в виде перестановок на конечном множестве из 54 цветных сегментов граней, а пространственные вращения слоев — как композиции данных перестановок.

В практической части нашей работы был спроектирован и описан алгоритм синтеза последовательности сборки. Высокая избирательность алгоритма и минимизация его влияния на уже собранные элементы куба были достигнуты за счет декомпозиции сложных многоэлементных циклов в систему перекрывающихся 3-циклов.

Проведенный асимптотический анализ показал, что алгоритм обладает линейной пространственной сложности $O(N)$ и линейной временной сложностью $O(N)$.

Список литературы

- [1] Винберг, Э. Б. Курс алгебры / Э. Б. Винберг. — Москва : МЦНМО, 2013. — 590 с. — ISBN 978-5-4439-0209-8.
- [2] Дубровский, В. Математика волшебного кубика / В. Дубровский // Наука и жизнь. — 1982. — № 8. — С. 138–143.
- [3] Кормен, Т. Х. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн ; пер. с англ. — 3-е изд. — Москва : Вильямс, 2013. — 1328 с.
- [4] Кострикин, А. И. Введение в алгебру. Часть 1. Основы алгебры / А. И. Кострикин. — Москва : ФИЗМАТЛИТ, 2012. — 272 с. — ISBN 978-5-9221-1422-5.
- [5] Седжвик, Р. Алгоритмы на C++ / Р. Седжвик ; пер. с англ. — Москва : Вильямс, 2011. — 1056 с.
- [6] Страуструп, Б. Язык программирования C++ / Б. Страуструп ; пер. с англ. — 4-е изд. — Москва : Бином, 2015. — 1136 с.
- [7] Joyner, D. Adventures in Group Theory: Rubik's Cube, Merlin's Machine, and Other Mathematical Toys / D. Joyner. — 2nd ed. — Baltimore : Johns Hopkins University Press, 2008. — 328 p.
- [8] Singmaster, D. Notes on Rubik's "Magic Cube" / D. Singmaster. — 5th ed. — Hillside : Enslow Publishers, 1981. — 71 p.